

Edge Negotiator — Defense Notes (point-by-point, lay terms)

Short points, grouped, so you can answer anything about the architecture and what was done. Pair with `REPORT.md` (narrative) and `results/` (numbers).

A. The problem and the claim

1. A junction's lights normally run on a fixed math rule; we test if a small local AI can run them as well or better.
2. Two halves: H1 = does the AI drive traffic well (delay), H2 = can we prove afterward what it did (audit).
3. It is a feasibility + scale study: the question is "is it even possible, and with how small a model," not "ship a product."
4. A negative or a limitation is an allowed, valid result. We never fake a win.
5. Real corridor: Euston Road A501, London, 4 signalised junctions, loaded into the SUMO traffic simulator.

B. The baseline we compete against (MaxPressure)

6. MaxPressure is the standard research controller: each decision it serves the direction with the highest "pressure."
7. Pressure = (cars waiting to enter a movement) minus (cars stuck on the road they would exit onto).
8. It is proven to maximise throughput (keep cars moving) under idealised assumptions.
9. Its weaknesses we exploit: it only sees the current queue size, is memoryless about how long cars have waited, and can switch phases too eagerly (wasted start/stop time).
10. It is deterministic (same input, same output), needs no training, and is a genuinely hard bar.

C. The controller architecture (how one decision flows)

11. The controller is `HybridController`: "the SLM proposes, the MaxPressure shield disposes."
12. Every ~10 seconds (a "decision interval") each junction decides which green phase to serve next.
13. Step 1 event-gate: if the junction is nearly empty, skip the AI and let MaxPressure handle it (saves compute).
14. Step 2: otherwise, build a short text prompt of the junction state and ask the SLM for a phase index.
15. Step 3 shield: if the AI's answer is valid, use it; if it is empty, out-of-range, or nonsense, fall back to MaxPressure's choice.
16. Step 4 anti-starvation: a fairness override forces a direction that has been skipped too many times, so nobody waits forever.
17. Step 5: the chosen phase is served; the light physically switches (with a yellow clearance) or holds.
18. Consequence: the AI can never be worse than "just MaxPressure" on validity; it can only try to improve delay.
19. Non-AI junctions (or empty ones) simply run MaxPressure.
20. "Phase" = one legal green configuration (e.g. north-south goes). Choosing a phase = choosing who moves.

D. The AI (SLM) and how we talk to it

21. SLM = Small Language Model, 0.5 to 4 billion parameters, run locally via Microsoft Foundry Local, no cloud, no internet.
22. We send a system prompt (its role + rules) and a user prompt (the current junction numbers), temperature 0 (deterministic).
23. It must reply with strict JSON like `{"phase": 1}`; we parse it and range-check it.
24. Any parse failure or invalid index returns "None," which triggers the shield fallback. Failures never crash the sim.
25. Accountability comes from the signed log, never from the AI explaining itself.

E. The three information settings ("configs")

26. myopic: the AI sees ONLY its own junction's per-phase queues. Short-sighted; the hardest, most honest case.
27. +coordination: the AI also gets a signed note from neighbours about traffic heading its way.
28. +prediction: the AI also gets an estimate of cars approaching but not yet stopped (a lever MaxPressure lacks).
29. We sweep all three to see which information actually helps.
30. Honest finding: on our simple 4-junction line, +coordination changed zero decisions (the neighbour notes never fired). Reported as a null, not a benefit.

F. How the AI beats MaxPressure (the core result)

31. Headline: under heavy demand, the smallest model (qwen2.5-0.5b), myopic, gave 235s mean delay vs MaxPressure's 314.55s, about 25% lower.
32. Why possible: MaxPressure is throughput-optimal, not delay-optimal, and its eager switching wastes time; the AI (shield-checked) makes slightly better phase choices for delay.
33. The metric is "mean network delay": average time lost across ALL cars that entered, not just finishers (so you cannot cheat by stranding slow cars).
34. Verdicts vs baseline: "beats" = >2% lower delay, "matches" = within 2%, "loses" = >2% higher.
35. Scale threshold result: the smallest model at the simplest config already beats. That is the strongest possible answer to "at what scale is it possible."
36. Bigger is not better here: phi-4-mini (largest usable) only matched plain; the tiny qwen models beat. A genuinely interesting, honest finding.

G. The SOTA "delay-aware" experiments (v1, v2)

37. We read the literature (LLMLight, EvolveSignal, etc.; saved in `01-research/beating-max-pressure.md`) for ways to beat MaxPressure.
38. Two levers MaxPressure lacks, both still myopic: (a) waiting-time priority (serve who has waited longest), (b) switching hysteresis (do not switch unless clearly better, to cut stops).
39. delay-aware v1: give the AI the queue AND the accumulated waiting time, tell it to minimise waiting and avoid needless switching.
40. v1 result: lifted phi-4-mini from a tie to a win (315s to 270s), improved qwen3-0.6b, but hurt the already-strong small models. A mixed, honest result.
41. delay-aware v2: pre-compute one "delay score" and ask the AI to just pick the highest (the research said small models are bad at arithmetic).
42. v2 result: net worse than v1 (dropped phi-4-mini and qwen3-1.7b to losing). Recorded as an honest negative; we did NOT keep tuning on one seed (that would be cheating).

43. Conclusion: prompt tricks give uneven gains; a uniform win across all models would need lightweight fine-tuning (future work). The headline stands on the plain result (point 31).

H. Models and latency

44. Models tested: qwen2.5-0.5b/1.5b (older), qwen3-0.6b/1.7b/4b (newer, 2025), phi-4-mini, phi-4-mini-reasoning.
45. qwen3 models default to writing long reasoning first (~7s per decision, too slow); their official `/no_think` switch drops that to ~0.3s. We enable it automatically for qwen3 only.
46. Decision latency for the usable models: 0.29 to 0.80 seconds, comfortably inside the ~10s a real junction allows.
47. phi-4-mini-reasoning is "latency-gated": it takes ~8.5s per decision (measured), too slow to sweep at scale, so it is honestly skipped with the measured number, not faked.

I. The trust half: the audit ledger

48. Every message, sighting, and decision is appended to an append-only "AuditLog."
49. Each junction has its own cryptographic identity (an Ed25519 key) and signs its own records, so a decision is provably attributable to a KEY (never a named person).
50. Records are hash-chained: each contains a fingerprint of the previous one, so inserting, deleting, or reordering any record breaks verification.
51. `verify_chain()` recomputes the whole chain and returns false if anything was altered.
52. A batch of records folds into one short "Merkle root" fingerprint; an "inclusion proof" proves one specific record is in that batch without revealing the others.
53. The published root is "salted" (scrambled with a secret) so it commits completeness without leaking the pseudonymous identities; erase the salt and it becomes unlinkable (privacy, §7.7).
54. "Quorum anchor": publish the root to at least 2 independent outside witnesses (e.g. a public transparency log plus a small blockchain node) so an operator cannot later show two different histories.
55. "Cross-auditor": an independent reader checks the witnesses agree; if two disagree that is "equivocation" and it is flagged.
56. The audit runs LIVE alongside the controller on every experiment arm, not as a mock.

J. The accident-reconstruction demo (D-accident, passes 8/8)

57. We script a crash: a compromised but approved neighbour signs a FAKE emergency-vehicle claim that hijacks the lights, then a REAL emergency vehicle is sensed locally.
58. From the verified log ALONE, the system reconstructs WHAT happened (the exact sequence of phases actually served).
59. It reconstructs HOW (which signed inputs drove each decision and which safety policies fired).
60. It reconstructs WHICH KEY drove the causal decision: the fake claim is attributed to the specific compromised key; the real vehicle is a keyless local sighting.
61. Tamper test: we swap in a forged signature that still passes the hash-chain, and the system STILL refuses to attribute, because a per-sender signature check catches it. "A tamper that survives the chain is caught anyway."
62. Critical earlier bug (now fixed): the log used to record the phase the AI PROPOSED, not the phase actually SERVED after the fairness override. That would have corrupted every reconstruction. Caught by the adversarial reviewer, fixed, re-verified.

K. The anchor (D-anchor, honest boundary)

- 63. We built the salted-root + 2-witness + cross-audit mechanism and proved it catches a planted "two different histories" (equivocation) attack.
- 64. We did NOT write to the real public transparency log (that would permanently pollute a shared public service with test data) and did not stand up the blockchain node.
- 65. So the run is honestly labelled "not externally anchored": the mechanism works, the public non-equivocation claim is withheld. This is the spec's own allowed honest outcome.
- 66. It also reconciles a per-signer count (how many records each key signed) as a process check.

L. The policy rulebook (`ground_rules.yaml`)

- 67. Authorised entities: ambulance, fire, police (may request signal preemption), maintenance (may close a lane but NEVER preempt), civilian (no priority).
- 68. Legitimacy criteria B1 to B5: identity (approved key), corroboration (an independent junction also saw it), local sensing, plausibility (physically possible counts), temporal consistency (timing makes sense).
- 69. Decision policies P1 to P8: condition to outcome. Key ones: P2 bad/absent key to reject; P3 a single uncorroborated claim can NEVER force a green (anti-phantom); P4 corroborated by ≥ 2 keys to legitimate; P8 unknown vehicle or missing data to UNKNOWN (conservative reject).
- 70. Attack taxonomy: invalid-id, signal-tampering (a maintenance key posing as an ambulance), phantom (a claim for a non-existent vehicle), count-inflation, missing-metadata, contradictory-signals, replay.

M. Multi-authorised-vehicle authorisation (new classifier)

- 71. We built a deterministic classifier that reads the entity rules and decides, for a claimed vehicle, whether it may preempt.
- 72. It outputs LEGITIMATE / SPOOFED_OR_FAULTY / UNKNOWN and names the policy and attack category it maps to (auditable).
- 73. Maintenance is NEVER granted preemption; a works key claiming to be an ambulance is flagged as signal-tampering; unknown vehicle or missing data resolves to UNKNOWN.
- 74. Evaluated on a balanced 26-case set across every attack family, scored with precision, recall, false-positive rate, false-negative rate.
- 75. Result: perfect classification and ZERO "dangerous false grants" (no illegitimate claim ever got a green). It is a rule gate, so perfection is expected; the value is proving full attack coverage.

N. The fault / origin channel (assessment)

- 76. After an incident, `fault_report` verifies the log (chain + signatures), then classifies the ORIGIN of a flagged event.
- 77. It names a KEY or situation (attacker-key / sensor-fed-spoof / legitimate / unknown), never a person, never a verdict.
- 78. It emits an "evidence pack" of only mechanically-true facts (no accusation, no confidence score, no "lie"), with a header saying it is not a determination of legal fault.
- 79. If verification fails at any layer, it REFUSES with "unknown" rather than guessing.

O. The audit's measured blind spot (D6, Experiment D)

- 80. Honest self-criticism: because the attack controls the light phase, and the phase controls which honest sensors can see a problem, an insider can open a "coverage desert."
- 81. In that desert, a carefully bounded fake (kept below the detection threshold) is neither seen live nor leaves a detectable trace. That is the self-referential coupling.
- 82. We demonstrate it as a mechanism, calibrate the detection threshold independently, and show escape rises when coverage is phase-locked vs evenly spread.
- 83. We state the exact four conditions under which it CANNOT happen (dense coverage + honest keys + cross-audited quorum + no operator omission).
- 84. The severe statistical version of this claim is honestly gated: it needs real documented sensor placement, which we do not have, so it is labelled "protocol, not a passed control."

P. The AI legal-note experiment (Job A, D5, honest negative)

- 85. A separate idea: have the AI write a citation-grounded legal note about an incident using a UK traffic-law knowledge base (18 statute/case rules).
- 86. Honest measured result: a plain non-AI template BEATS the AI on citation correctness and false-citation rate. Reported as a capability limitation.
- 87. This note is non-evidential and never enters the evidence pack; it is a support artifact, not a fault determination.

Q. Grading criteria

- 88. Layer 1: the delay verdict (beats/matches/loses, 2% tie-band) plus latency under the 10s window plus audit intact.
- 89. Layer 2: the "§12 gates," a checklist of acceptance criteria (D1..D8, D-lat, D-anchor, D-accident, D-H1-perf, D-H1-scale, D-sec, D-SLM-kill).
- 90. Each gate passes either by a real run OR by an "honest gate" (recording exactly what external data is missing rather than faking it).
- 91. Passed by real run: D-H1-perf (pilot), D-H1-scale, D-accident, D-anchor (mechanism), D-lat, the vehicle authorisation, D7 fairness.
- 92. Honest pilots / negatives: D5 (template wins), D4 (pilot done), D8 (demand sweep pilot), D6 (mechanism, inferential gated).
- 93. Gated on external data: all statistical-significance ($n \geq 30$ "powered") claims.

R. Honesty machinery (why this is trustworthy)

- 94. PILOT / $n=1$: most traffic results are one run, one seed. We explicitly claim no statistical significance from them.
- 95. The §8 demand gate: our demand is calibrated to daily-average real counts, not hour-by-hour, so any significance claim is held back until proper hourly data lands.
- 96. The "battery": before any work is accepted, a SEPARATE adversarial AI reviewer (that did not write the code) tries to break it; zero fatal + zero major findings required to proceed.
- 97. builder != examiner: the agent that builds never grades its own work; reviewers run synchronously so they cannot silently vanish.
- 98. Every phase was committed only after passing its battery; 745 automated tests pass.

S. Bugs the reviewers caught (shows the process works)

- 99. FATAL audit bug: the log recorded the proposed phase, not the served phase after the fairness override. Fixed and re-verified.
- 100. FATAL silent-degradation bug: if the AI service died mid-sweep, the controller silently fell back to MaxPressure, and for myopic that is identical to the baseline, so it would have falsely reported a "match." We added a guard that flags "AI authored zero real decisions" as degraded, not a pass.
- 101. Third bug: a delay-aware reader called a SUMO function that does not exist, silently feeding the AI zeros. Caught by checking the real API and fixed.

T. Data provenance and demand honesty

- 102. Demand magnitude and vehicle mix come from real DfT (UK Dept for Transport) count data on the A501 (count points 18077, 56815).
- 103. Originally the hourly shape and direction split were assumed (a peak-hour fraction of 0.085).
- 104. We downloaded the DfT raw manual-count release and extracted the MEASURED hourly profile (2024 survey: peak hour 18:00, 2461 veh/h, real 48/52 direction split, real mix).
- 105. We wired a measured-demand file (`base_hourly.rou.xml`); this removes the hourly-shape assumption but is still a single survey day, so still a pilot.
- 106. Still missing for full significance: time-resolved TfL hourly + junction turning counts. That is data, not code, and is the one genuine external blocker.

U. Engineering

- 107. The whole thing is a Python codebase with a large automated test suite (745 passing).
- 108. The traffic simulator is SUMO 1.26 driven live via its TraCI control interface.
- 109. The repo is "greenfield-clean": it reads as one coherent design, with iteration history only in git.
- 110. Certain critical files are "hash-pinned" in the spec so they cannot silently change; the one deliberate change (adding the vehicle rules) re-pinned carefully.

V. Likely questions and crisp answers

- 111. "Did the AI really beat the standard method?" Yes, on delay, under heavy demand, even at the smallest model size; but it is a single-seed pilot, not yet a significance claim.
- 112. "Isn't MaxPressure optimal?" For throughput, yes; for delay it is not, and it is memoryless and switches eagerly, which is the gap the AI exploits.
- 113. "How is the AI kept safe?" A deterministic MaxPressure shield checks every AI proposal; invalid answers fall back to MaxPressure, so the AI can never be worse on validity.
- 114. "What stops the AI hallucinating a bad move?" The shield plus range-checking plus anti-starvation; and accountability is from the signed log, never the AI's self-explanation.
- 115. "What does the ledger actually guarantee?" That the record of decisions is tamper-evident (chain), attributable to a key (signatures), complete (Merkle), and privacy-preserving (salt); with anchoring it also resists showing two different histories.
- 116. "What if someone forges a signature?" The accident demo proves a forged signature that still passes the chain is caught by the per-sender identity check, and attribution is refused.
- 117. "Why only local models?" The whole point is on-device (no cloud): privacy, no network dependency, and it must run inside a ~10s real-time budget.
- 118. "Why is coordination not helping?" On this simple 4-junction line the neighbour signal never fired; we report that honestly rather than claim a benefit.

119. "What is the biggest limitation?" Everything traffic-related is a daily-demand pilot; the powered statistical claims wait on real hourly Tfl data.
120. "What is genuinely novel vs prior work?" Not the crypto (credited to Certificate Transparency) and not the LLM-control idea (credited to LLMLight); the contribution is putting them together on a real corridor with an honest feasibility + scale answer and an honestly-reported blind spot.
121. "Could the result be a fluke of the shield doing the work?" We measure how often the AI actually authored the served decision (not the shield); in the winning runs the AI authored a real share, and degraded all-shield runs are flagged, not counted.
122. "What would you do next?" Get hourly Tfl data to run powered $n \geq 30$ experiments, and try lightweight LoRA fine-tuning to make the smallest models win uniformly.